

Instituto Politécnico Nacional

Escuela Superior de Cómputo / Unidad Profesional
Interdisciplinaria

Ingeniería en Inteligencia Artificial (Plan de Estudios 2020)

Fundamentos Teóricos y Arquitectura de los Sistemas Gestores de Bases de Datos

Una aproximación sistemática desde el procesamiento de archivos hasta
los paradigmas contemporáneos en Inteligencia Artificial

Unidad de Aprendizaje: Bases de Datos

Unidad Temática I: Introducción a las Bases de Datos y Modelado Conceptual

Autor: Cesar Javier Martinez Ruiz

Número de Boleta: 2024630871

Grupo: 3BV1

Fecha: Septiembre de 2024

1. Dato, Información y Base de Datos: Conceptualización y Análisis Comparativo

En la ciencia de la computación contemporánea, la distinción entre dato e información constituye el pilar epistemológico para el diseño de cualquier sistema de cómputo. Un **dato** representa una representación simbólica formal, no evaluada, atómica y desprovista de contexto contextual intrínseco; puede manifestarse como una cifra numérica, una cadena de caracteres alfanuméricos, una señal discreta o un vector en un espacio euclidiano multidimensional (Date, 2004). Por su parte, la **información** surge cuando los datos son estructurados, procesados, depurados y contextualizados dentro de un marco semántico relacional que permite al receptor humano o algorítmico reducir la incertidumbre, extraer significado inferencial y fundamentar la toma de decisiones (Elmasri & Navathe, 2016).

Para conceptualizar rigurosamente una **base de datos**, es imperativo contrastar las formulaciones teóricas de los principales tratadistas del área:

Definición A: Para Elmasri y Navathe (2016), una base de datos es «una colección lógicamente coherente de datos con algún significado inherente, diseñada, construida y poblada con datos para un propósito específico, la cual representa algún aspecto del mundo real (a menudo denominado mini-mundo o Universo de Discurso)» (p. 4). Los autores enfatizan que una disposición aleatoria de datos nunca constituye una base de datos; exige una cohesión lógica predeterminada que refleje fielmente el estado dinámico del universo modelado.

>

Definición B: Por otro lado, Silberschatz, Korth y Sudarshan (2020) definen una base de datos en conjunción con su sistema de control como «una colección de datos interrelacionados y un conjunto de programas para acceder a dichos datos [...], cuyo objetivo primordial es proporcionar un entorno que sea a la vez conveniente y eficiente para recuperar y almacenar información de la base de datos» (p. 1).

Análisis comparativo de ambas perspectivas

Al confrontar ambas definiciones se aprecian matices pedagógicos y ontológicos determinantes:

1. Enfoque ontológico vs. Enfoque operacional: Mientras que Elmasri y Navathe (2016) abordan la base de datos desde una perspectiva ontológica y semántica —resaltando el concepto de *mini-mundo* y la coherencia lógica interna independiente del software—, Silberschatz et al. (2020) adoptan una visión orientada a la ingeniería de sistemas, donde el valor de la base de datos es indisociable del software gestor que garantiza el acceso eficiente, la concurrencia y la reducción de costos de cómputo.

2. Propósito del modelado: Para Elmasri y Navathe, la meta es la fidelidad de representación de la realidad observada; para Silberschatz et al., la meta es la conveniencia, el rendimiento y la abstracción algorítmica para los desarrolladores y usuarios finales. Ambas visiones no se contraponen, sino que se complementan: una base de datos debe poseer coherencia semántica en su esquema conceptual para que los algoritmos de acceso puedan operar de forma óptima y predecible.

2. Fundamentos de las Bases de Datos: Del Enfoque de Archivos al SGBD

Con anterioridad a la consolidación de los Sistemas Gestores de Bases de Datos (SGBD o DBMS, por sus siglas en inglés *Database Management System*) en la década de 1970, el almacenamiento persistente empresarial operaba bajo el **enfoque tradicional de procesamiento de archivos**. En dicho paradigma, cada departamento o aplicación de software mantenía sus propios archivos planos estructurados secuencialmente (como registros en COBOL o ficheros binarios). Cada programa de aplicación contenía codificada de manera estática la definición física de los registros y los algoritmos de búsqueda (García-Molina et al., 2009).

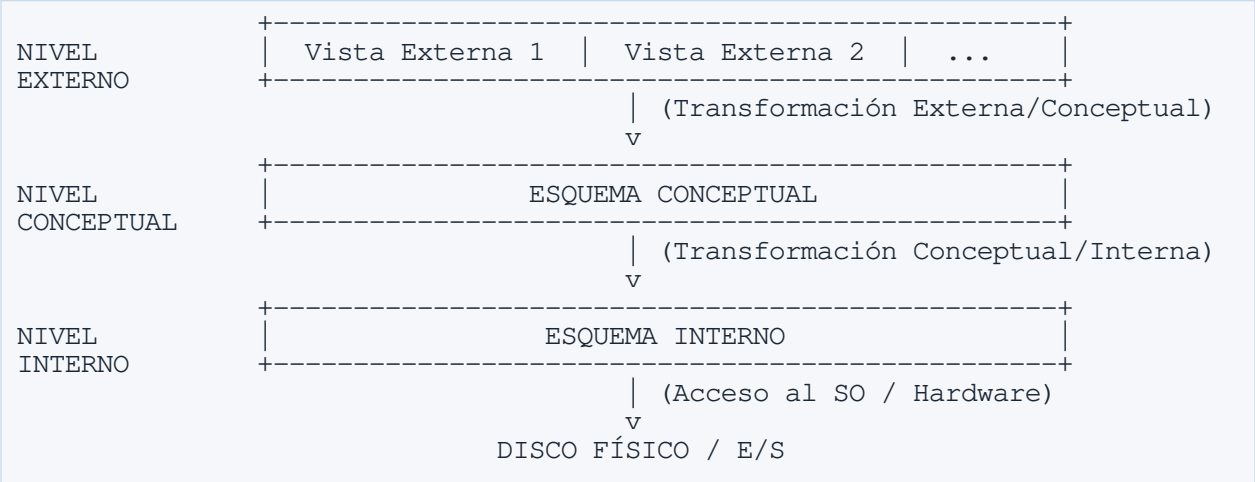
Este modelo primitivo colapsó debido a limitaciones estructurales críticas:

1. **Redundancia e inconsistencia de datos:** Al carecer de un repositorio centralizado, la misma información (por ejemplo, el domicilio de un alumno) se duplicaba en múltiples archivos disjuntos. Cuando ocurría una modificación en un departamento, la omisión de propagación a los demás archivos generaba inconsistencia de datos, destruyendo la fiabilidad del sistema (Ramakrishnan & Gehrke, 2003).
2. **Dificultad en el acceso a los datos:** Cualquier consulta no anticipada por los programadores originales (ej. «listar los alumnos con promedio superior a 9 que adeudan material bibliográfico») requería escribir un nuevo programa desde cero para recorrer manualmente los archivos, demorando días lo que debería resolverse en segundos.
3. **Aislamiento de datos y dispersión de formatos:** Los archivos eran creados en formatos binarios incompatibles por diferentes lenguajes o programadores, imposibilitando la interoperabilidad o el cruce de variables (Silberschatz et al., 2020).
4. **Problemas de integridad y dispersión de reglas:** Las restricciones del negocio (ej. un saldo no puede ser negativo) estaban incrustadas en el código fuente de los programas de aplicación. Si las políticas cambiaban, era indispensable rastrear y modificar decenas de scripts individuales, propiciando errores humanos.
5. **Anomalías en el acceso concurrente:** Si dos programas intentaban debitar o modificar el mismo registro de un archivo simultáneamente sin un gestor transaccional de bloqueos (*locks*), se producían sobreescrituras desastrosas (*lost updates*).
6. **Problemas de seguridad y atomicidad:** Los sistemas de archivos de los sistemas operativos ofrecían permisos rudimentarios (lectura/escritura a nivel archivo completo), impidiendo otorgar acceso granular por columnas, filas o roles, además de carecer de mecanismos de reversión (*rollback*) ante apagones repentinos durante transacciones monetarias complejas.

El SGBD surgió precisamente para desacoplar las aplicaciones de los datos físicos, centralizando la definición, manipulación, seguridad, concurrencia y recuperación en una capa intermedia estandarizada (Date, 2004).

3. La Arquitectura ANSI-SPARC: Origen, Propósito y Esquemas

A mediados de los años 70, la proliferación de sistemas de bases de datos incompatibles exigía un marco de referencia teórico estándar. En 1975, el Comité de Planificación y Requisitos de Estándares de la Asociación Nacional Estadounidense de Estándares (ANSI/SPARC, *American National Standards Institute, Standards Planning and Requirements Committee*) publicó una propuesta arquitectónica seminal con el propósito primordial de garantizar la **independencia de datos** y desacoplar las vistas de los usuarios de la implementación física del hardware (Date, 2004; Elmasri & Navathe, 2016).



La arquitectura propone tres niveles de abstracción claramente delimitados:

1. Nivel Externo (Vistas de Usuario): Constituye el nivel más próximo a los usuarios finales y aplicaciones cliente. Describe únicamente la porción de la base de datos relevante para un grupo de usuarios específico, ocultando el resto de los datos y la complejidad técnica. Por ejemplo, en una universidad, la interfaz de un estudiante solo muestra su historial académico y sus préstamos activos, mientras que el departamento de finanzas accede a estados de cuenta y cobranza, manteniendo cada uno su propia *vista externa*.

2. Nivel Conceptual (Esquema Global Lógico): Es la representación abstracta y unificada de la estructura global completa de la base de datos para toda la organización. Oculta por completo los detalles de almacenamiento físico e infraestructura de hardware, concentrándose en describir las entidades, atributos, tipos de datos, relaciones lógicas y restricciones de integridad del dominio (Silberschatz et al., 2020). Es el nivel donde se formalizan los modelos relacionales (tablas, claves primarias y foráneas).

3. Nivel Interno (Esquema Físico): Es la representación física del almacenamiento en disco. Describe pormenorizadamente cómo se organizan los registros en los bloques del sistema operativo, las estructuras de direccionamiento, los tipos de archivos (secuenciales, dispersos o *heap*), las técnicas de compresión y cifrado, y la implementación de índices auxiliares (como árboles B+ o tablas hash) para acelerar la recuperación de bloques de memoria (Garcia-Molina et al., 2009).

4. Independencia de Datos, Lenguajes de un SGBD y Módulos Componentes

4.1. Independencia de Datos

La mayor contribución teórica de la arquitectura ANSI-SPARC es la capacidad de modificar la definición de un esquema en un nivel sin tener que alterar los esquemas de los niveles superiores. Se divide en dos dimensiones fundamentales:

* **Independencia lógica de datos:** Es la inmunidad de las aplicaciones cliente y esquemas externos frente a cambios en el esquema conceptual (ej. añadir una nueva entidad, agregar un atributo o reestructurar una tabla en dos mediante normalización). Las vistas externas preexistentes se preservan mediante redefinición de transformaciones lógicas sin necesidad de reescribir las consultas ni los programas de usuario (Elmasri & Navathe, 2016).

* **Independencia física de datos:** Es la capacidad de modificar el esquema interno sin necesidad de alterar el esquema conceptual ni las aplicaciones externas. Esto permite a los administradores (DBA) reorganizar archivos en disco, migrar a discos de estado sólido (NVMe), cambiar tamaños de bloques o crear y destruir índices de búsqueda para optimizar el rendimiento sin que los usuarios finales perciban alteración alguna en la lógica del sistema (Ramakrishnan & Gehrke, 2003).

4.2. Lenguajes de un SGBD

Para interactuar con los diferentes niveles, un SGBD provee lenguajes especializados que en sistemas relacionales convergen en SQL:

4.3. Módulos Componentes de un Motor SGBD

Un SGBD moderno se estructura internamente en subsistemas cooperativos (Silberschatz et al., 2020):

1. **Procesador de Consultas (*Query Processor*):** Incluye el *parser* (analizador léxico y sintáctico), el verificador semántico de catálogos y el **Optimizador de Consultas (*Query Optimizer*)**, el cual traduce una expresión relacional declarativa en el plan de ejecución físico de menor costo computacional basándose en estadísticas de disco.
2. **Gestor de Almacenamiento (*Storage Manager*):** Interfaz entre el procesador de consultas y el subsistema de disco del sistema operativo. Administra la asignación de espacio y gestiona el *Buffer Manager* o pool de páginas en memoria RAM.
3. **Gestor de Transacciones y Control de Concurrencia:** Garantiza las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) implementando protocolos de bloqueo en dos fases (2PL), control de concurrencia multiversión (MVCC) y gestión del registro de transacciones (*Write-Ahead Logging* o WAL) para recuperación automática ante desastres.

5. Taxonomía de las Bases de Datos y Casos de Uso Reales

La diversificación de patrones de acceso y volúmenes masivos de información en la era digital condujo al auge del movimiento *Polyglot Persistence*, donde conviven diversos modelos de datos:

• **Tipo de Base de Datos** • **Estructura de Datos Central** • **Fortalezas Técnicas** • **Caso de Uso Real**

• **Relacionales (RDBMS)** • Tablas formadas por tuplas y atributos; normalización e integridad referencial estricta. • Transaccionalidad ACID rigurosa, consistencia inmediata, consultas complejas mediante álgebra relacional en SQL. • **Sistemas de Core Bancario y Contabilidad:** Gestión de transferencias y cuentas de débito donde la inconsistencia monetaria es inaceptable (ej. PostgreSQL, Oracle).

- **Documentales** • Documentos semiestructurados jerárquicos (típicamente JSON, BSON o XML) con esquemas dinámicos. • Flexibilidad de esquema, escalabilidad horizontal mediante sharding, anidación de estructuras complejas. • **Catálogos de Comercio Electrónico:** Plataformas como Amazon o MercadoLibre, donde productos de distintas categorías poseen atributos heterogéneos (ej. MongoDB).
- **Clave-Valor (Key-Value)** • Pares atómicos indexados por una clave única accesible en tiempo constante $O(1)$. • Latencia ultrabaja de lectura y escritura en memoria volátil o disco, alta velocidad de particionamiento. • **Gestión de Sesiones y Caché en Tiempo Real:** Almacenamiento de tokens de autenticación web y carritos de compra temporales de alta concurrencia (ej. Redis, DynamoDB).
- **Familias de Columnas (Wide-Column)** • Filas bidimensionales compuestas por claves y familias de columnas dispersas (*sparse*). • Optimizado para agregaciones masivas y escrituras secuenciales continuas de Big Data distribuido. • **Telemetría y Registro Masivo de Eventos IoT:** Sensores industriales de turbinas que registran miles de métricas concurrentes por segundo (ej. Apache Cassandra).
- **Bases de Datos de Grafos** • Nodos (entidades), aristas (relaciones dirigidas) y propiedades asociadas (*LPG*). • Recorridos de relaciones de alta profundidad (*index-free adjacency*) sin costosas operaciones JOIN. • **Detección de Fraude Financiero y Redes Sociales:** Mapeo de anillos de lavado de dinero o grafos de recomendación de amigos en LinkedIn o Meta (ej. Neo4j).
- **Series de Tiempo (Time-Series)** • Registros optimizados cronológicamente con marcas de tiempo (*timestamps*) inmutables. • Compresión masiva de datos temporales, retención por ventanas y agregaciones temporales ultrarrápidas. • **Monitorización de Mercados Financieros y Sistemas:** Cotizaciones de acciones en Wall Street o métricas de servidores en tiempo real (ej. TimescaleDB, InfluxDB).
- **Espaciales / Geoespaciales** • Tipos de datos geométricos (puntos, polígonos, multilíneas) e índices espaciales (R-Tree, GiST). • Cálculo eficiente de distancias geodésicas, intersecciones poligonales y áreas de influencia. • **Plataformas de Movilidad y Logística:** Uber o Didi calculando el conductor más cercano y optimizando rutas sobre mapas vectoriales (ej. PostGIS).
- **Vectoriales (Vector DB)** • Vectores densos de alta dimensión (*embeddings*) producidos por redes neuronales profundas. • Búsqueda aproximada de vecinos más cercanos (ANN) mediante distancias de coseno o euclidianas a alta escala. • **Sistemas de Búsqueda Semántica y RAG para LLMs:** Motores que recuperan fragmentos de texto contextual para ChatGPT basándose en similitud conceptual (ej. Milvus, Pinecone).

6. Clasificación Sistemática de los Sistemas Gestores

Los SGBD se clasifican formalmente de acuerdo con cinco criterios de ingeniería (Elmasri & Navathe, 2016; Silberschatz et al., 2020):

1. **Por el modelo de datos lógico:** Relacionales (SQL), NoSQL (documentales, clave-valor, grafos, columnas), Objeto-Relacionales (PostgreSQL), Orientados a Objetos y Multimodelo.
2. **Por el número de usuarios:**

Monousuario:* Diseñados para una sola estación de trabajo local (ej. SQLite empotrado en aplicaciones móviles de Android/iOS o Microsoft Access).

Multiusuario:* Diseñados para admitir miles de conexiones concurrentes gestionando bloqueos transaccionales y aislamiento (ej. PostgreSQL, Oracle Database).

3. Por la distribución física de sitios:

Centralizados:* Toda la base de datos y el motor ejecutan en una sola máquina física o servidor maestro.

Distribuidos:* Los datos residen fragmentados o replicados en múltiples nodos geográficos interconectados por red (ej. Google Spanner, CockroachDB), administrando consistencia distribuida (Teorema CAP).

4. Por el propósito y carga de trabajo:

Propósito General (OLTP - Online Transaction Processing):* Optimizados para miles de transacciones pequeñas, lecturas y escrituras atómicas concurrentes.

Propósito Analítico (OLAP - Online Analytical Processing) y Almacenes de Datos (*Data Warehouses*):* Optimizados para consultas complejas de agregación sobre millones de registros históricos (ej. Snowflake, ClickHouse).

5. Por el modelo de licenciamiento:

Código Abierto (*Open Source*):* Licencias libres como PostgreSQL (licencia propia tipo MIT/BSD) y MySQL/MariaDB (GPL).

Propietarios / Comerciales:* Licencias pagadas por núcleo de CPU o suscripción empresarial como Oracle Database, Microsoft SQL Server e IBM Db2.

7. Bases de Datos en los Flujos de Trabajo de Inteligencia Artificial

En el paradigma de la Inteligencia Artificial y la Ciencia de Datos moderna, las bases de datos han dejado de ser meros almacenes pasivos para convertirse en el núcleo de la infraestructura de cómputo y entrenamiento:

7.1. Almacenamiento y Versionado de Conjuntos de Entrenamiento

El entrenamiento de modelos de Aprendizaje Automático supervisado y Redes Neuronales Profundas requiere colecciones masivas de datos multimodales (texto, audio, imágenes y métricas estructuradas). Las bases de datos modernas garantizan la reproducibilidad experimental mediante esquemas de **versionado de datos** (*Data Versioning*) y arquitecturas *Lakehouse* (combinación de Data Lakes con capas transaccionales ACID como Apache Iceberg o Delta Lake). Esto asegura que un modelo reentrenado seis meses después utilice exactamente la misma instantánea inmutable de datos con la que fue validado originalmente.

7.2. Recuperación de Información y Arquitecturas RAG (*Retrieval-Augmented Generation*)

Los Grandes Modelos de Lenguaje (LLMs) adolecen de dos problemas congénitos: limitaciones en su ventana de contexto temporal y la propensión a alucinaciones factuales. Para subsanarlo, las arquitecturas RAG acoplan un SGBD como memoria externa de largo plazo. Cuando un usuario formula una pregunta, el sistema consulta en milisegundos la base de datos documental o relacional corporativa para extraer hechos fehacientes y alimentar dinámicamente el *prompt* de entrada del modelo, garantizando respuestas fidedignas y auditables.

7.3. Sistemas de Búsqueda por Similitud Vectorial

La búsqueda tradicional por concordancia léxica exacta (ej. sentencias `WHERE column LIKE '%palabra%'`) es incapaz de capturar la semántica conceptual. Las bases de datos vectoriales almacenan las representaciones vectoriales latentes (*embeddings*) generadas por encoders de Deep Learning (como BERT o CLIP). Mediante algoritmos de **Búsqueda Aproximada del Vecino más Cercano** (ANN, *Approximate Nearest Neighbor*) —tales como grafos jerárquicos navegables de mundo pequeño (*HNSW - Hierarchical Navigable Small World*) o índices de cuantización de productos (*IVF-PQ*)— el motor de base de datos puede comparar distancias geométricas (coseno, producto punto o distancia L2) en espacios de más de 1500 dimensiones en milisegundos, impulsando sistemas de recomendación semántica, detección de anomalías y reconocimiento biométrico.

Referencias Bibliográficas (Formato APA 7.^a Edición)

Date, C. J. (2004). An Introduction to Database Systems (8th ed.). Addison-Wesley.*

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.*

Garcia-Molina, H., Ullman, J. D., & Widom, J. (2009). Database Systems: The Complete Book (2nd ed.). Pearson Prentice Hall.*

Ramakrishnan, R., & Gehrke, J. (2003). Database Management Systems (3rd ed.). McGraw-Hill.*

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill.*